

RAPPORT SCIENTIFIQUE

# Optimisation d'un Agent Intelligent sur Taxi-v3

---

*Bruteforce, Monte Carlo, SARSA, Q-Learning et Deep Q-Learning :  
étude comparative sur l'environnement Gymnasium Taxi-v3*

|                            |                                                              |
|----------------------------|--------------------------------------------------------------|
| <b>Auteurs</b>             | Sabri Hammi, Marwan Meziane,<br>Jean Dubois, Robin Arzinière |
| <b>Projet de référence</b> | TAXI-DRIVER                                                  |
| <b>Environnement</b>       | Gymnasium Taxi-v3                                            |
| <b>Date</b>                | 2 juillet 2026                                               |

# Résumé

---

Ce rapport étudie la résolution de l'environnement **Taxi-v3** (Gymnasium) par apprentissage par renforcement **model-free**, en confrontant cinq approches : une baseline aléatoire (**Bruteforce**), une méthode épisodique par échantillonnage (**Monte Carlo**), deux méthodes de différence temporelle — l'une **on-policy** (**SARSA**), l'autre **off-policy** (**Q-Learning**) — et une approximation par réseau de neurones (**Deep Q-Learning**). Après formalisation du problème en Processus de Décision Markovien (500 états, 6 actions), chaque algorithme est implémenté, entraîné avec des hyperparamètres propres, puis évalué sur un protocole de test identique. Les résultats mesurés sur l'implémentation de référence montrent une hiérarchie claire : le Bruteforce échoue systématiquement (reward moyen  $\approx -770$ , 0 % de succès), tandis que les quatre méthodes de RL convergent toutes vers une politique sans pénalité. Le **DQN** obtient le meilleur reward (+8.44) mais au prix d'un entraînement 17,6 fois plus long que **SARSA** (+7.28 en 27 s contre 476 s) pour un gain marginal. L'analyse met en évidence le principe de parcimonie : pour un espace d'états discret de taille modeste, les méthodes tabulaires dominent l'approximation par réseau de neurones en efficacité. Le rapport se conclut par une discussion des limites du tabulaire face au fléau de la dimension, et des perspectives vers des environnements de conduite continus.

# Table des matières

---

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| Résumé .....                                                             | 2  |
| I Introduction, Contexte et Objectifs .....                              | 4  |
| I.1 Contexte du Projet .....                                             | 4  |
| I.1.1 L'environnement Taxi-v3 (Gymnasium) .....                          | 4  |
| I.1.2 Problématique métier et technique .....                            | 4  |
| I.2 Objectifs Spécifiques .....                                          | 4  |
| II Modélisation Mathématique (Le Framework MDP) .....                    | 5  |
| II.1 Formalisation en Processus de Décision Markovien .....              | 5  |
| II.1.1 Espace d'états $S$ .....                                          | 5  |
| II.1.2 Espace d'actions $A$ .....                                        | 5  |
| II.1.3 Signal de récompense $R$ .....                                    | 5  |
| III Présentation et Spécificités des Modèles Implémentés .....           | 7  |
| III.1 La Baseline : Algorithme Brute-force Naïf .....                    | 7  |
| III.2 L'Approche par Échantillonnage : Monte Carlo Épisodique .....      | 7  |
| III.3 Le Différentiel Temporel On-Policy : SARSA .....                   | 7  |
| III.4 Le Différentiel Temporel Off-Policy : Q-Learning .....             | 8  |
| III.5 L'Approche par Approximation Globale : Deep Q-Learning (DQN) ..... | 8  |
| IV Analyse des Hyperparamètres et Fine-Tuning .....                      | 9  |
| IV.1 Impact Théorique des Hyperparamètres .....                          | 9  |
| IV.2 Hyperparamètres réellement retenus par algorithme .....             | 9  |
| IV.3 Optimisation automatisée : Grid Search .....                        | 10 |
| IV.4 Analyse Avant vs Après Fine-Tuning .....                            | 10 |
| IV.5 Le CVaR comme indicateur de risque .....                            | 10 |
| V Benchmarking et Comparaison des Performances .....                     | 12 |
| V.1 Tableau Comparatif Global .....                                      | 12 |
| V.2 Courbes de convergence (données réelles d'entraînement) .....        | 13 |
| V.3 Analyse Croisée et Interprétation .....                              | 14 |
| VI Conception de l'Application Intégrée .....                            | 15 |
| VI.1 Architecture logicielle .....                                       | 15 |
| VI.2 Expérience Utilisateur (UX) .....                                   | 15 |
| VI.3 Supervision et déploiement .....                                    | 15 |
| VII Conclusion et Perspectives .....                                     | 17 |
| VII.1 Bilan des réalisations .....                                       | 17 |
| VII.2 Limites du Tabulaire : le Fléau de la Dimension .....              | 17 |
| VII.3 Perspectives .....                                                 | 17 |

# I Introduction, Contexte et Objectifs

---

## I.1 Contexte du Projet

### I.1.1 L'environnement Taxi-v3 (Gymnasium)

**Taxi-v3** modélise un problème de logistique urbaine miniature sur une grille de  $5 \times 5$  cases. Un taxi doit :

1. se déplacer jusqu'à la position d'un passager,
2. le prendre en charge (**Pickup**),
3. rejoindre l'une des quatre destinations balisées (R, G, Y, B),
4. le déposer (**Dropoff**).

Le réseau routier n'est pas une grille libre : des murs internes contraignent certains déplacements latéraux, ce qui interdit les trajets en ligne droite et force l'agent à apprendre des détours. C'est un problème à la fois petit (état discret, entièrement observable) et non trivial : le nombre d'actions possibles croît vite avec l'horizon, et une politique aléatoire ne converge jamais vers une solution correcte.

### I.1.2 Problématique métier et technique

La structure du problème — atteindre un point de collecte, transporter une charge, la livrer à un point de dépose, sous contrainte de coût par unité de temps — est directement transposable à des cas industriels réels : répartition de courses **Uber/VTC**, tournées de livraison du dernier kilomètre, routage d'entrepôt. Taxi-v3 sert ainsi de banc d'essai pédagogique pour la prise de décision séquentielle sous incertitude, avant d'attaquer des environnements de navigation plus réalistes.

## I.2 Objectifs Spécifiques

- Développer une **baseline naïve** (agent aléatoire) pour borner la performance en cas d'échec total.
- Implémenter et comparer **quatre paradigmes de Reinforcement Learning** sans modèle (Monte Carlo, SARSA, Q-Learning, DQN) sur un protocole d'évaluation identique.
- Concevoir une **application de pilotage interactive** exposant les modèles entraînés (inférence, benchmark visuel, comparaison d'algorithmes).
- Distinguer deux contraintes opérationnelles : un **mode « utilisateur »** qui expose tous les hyperparamètres pour l'expérimentation, et un **mode « time-limited »** qui applique une configuration pré-optimisée pour un usage rapide et reproductible.

## II Modélisation Mathématique (Le Framework MDP)

### II.1 Formalisation en Processus de Décision Markovien

Taxi-v3 est un MDP fini

$$(S, A, P, R, \gamma) \quad (1)$$

, entièrement observable et déterministe dans ses transitions (hors dynamique aléatoire de l'exploration  $\varepsilon$ -greedy côté agent).

#### II.1.1 Espace d'états $S$

Un état encode trois informations indépendantes :

$$|S| = \underbrace{5 \times 5}_{\text{position du taxi}} \times \underbrace{5}_{\text{état du passager}} \times \underbrace{4}_{\text{destination}} = 500 \quad (2)$$

- **Position du taxi** :  $5 \times 5 = 25$  cases de la grille.
- **État du passager** : 5 valeurs — à l'une des 4 stations (R, G, Y, B) ou déjà à bord du taxi.
- **Destination** : 4 valeurs (l'une des stations R, G, Y, B).

Soit  $25 \times 5 \times 4 = 500$  états discrets, tous représentables par un unique entier  $state \in [0, 499]$  — ce qui permet une Q-table dense de taille  $500 \times 6$  sans aucune approximation.

#### II.1.2 Espace d'actions $A$

6 actions discrètes :

| Id | Action  | Effet                                                |
|----|---------|------------------------------------------------------|
| 0  | South   | déplacement                                          |
| 1  | North   | déplacement                                          |
| 2  | East    | déplacement                                          |
| 3  | West    | déplacement                                          |
| 4  | Pickup  | charge le passager si le taxi est sur sa case        |
| 5  | Dropoff | dépose le passager si le taxi est sur la destination |

#### II.1.3 Signal de récompense $R$

| Événement                                 | Récompense |
|-------------------------------------------|------------|
| Pas de temps standard                     | -1         |
| Pickup / Dropoff illégal (échec critique) | -10        |
| Livraison réussie                         | +20        |

Ce barème façonne directement le comportement optimal : le coût -1 par pas pousse l'agent à minimiser la longueur du trajet (recherche du plus court chemin), la pénalité -10 décourage les actions de chargement hors contexte, et la prime +20 garantit qu'une politique gagnante reste positive même après plusieurs dizaines de pas.

**Reward shaping.** Le barème ci-dessus est celui, standard, fourni par Gymnasium et conservé tel quel dans l'implémentation de référence. Une piste de fine-tuning non explorée ici consisterait à augmenter la pénalité de pas relativement au bonus de succès pour accélérer la convergence sur des variantes à grille plus grande, où le ratio coût-de-trajet / prime-de-succès devient défavorable.

# III Présentation et Spécificités des Modèles Implémentés

## III.1 La Baseline : Algorithme Brute-force Naïf

Un agent sans mémoire ni apprentissage : à chaque pas, l'action est tirée uniformément dans  $A$  via `env.action_space.sample()`. Il n'existe aucune mise à jour de politique — c'est une marche aléatoire pure dans l'espace d'états.

**Espérance de gain et limites.** Sans biais de navigation, l'agent erre jusqu'à épuiser la limite de 200 pas (`truncated`) dans l'écrasante majorité des cas, chaque pas coûtant  $-1$  et chaque tentative de pickup/dropoff hors-contexte coûtant  $-10$ . Il ne converge jamais : le nombre d'actions nécessaires pour atteindre par hasard la séquence correcte (aller au passager  $\rightarrow$  pickup  $\rightarrow$  aller à la destination  $\rightarrow$  dropoff) croît de façon combinatoire avec la taille de la grille. C'est la définition même d'un plancher de performance : tout algorithme d'apprentissage doit strictement le dominer pour être jugé utile.

## III.2 L'Approche par Échantillonnage : Monte Carlo Épisodique

**Fonctionnement.** Contrairement aux méthodes de différence temporelle, Monte Carlo n'actualise  $Q(S, A)$  qu'**après la fin complète de l'épisode**, à partir du retour cumulé réellement observé  $G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$  :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[G_t - Q(S_t, A_t)] \quad (3)$$

L'implémentation utilise la variante **first-visit** : seule la première occurrence d'une paire  $(S, A)$  dans l'épisode contribue à la mise à jour.

**Spécificité sur Taxi-v3.** Cette dépendance à la terminaison de l'épisode est la principale faiblesse de la méthode sur ce problème : en tout début d'entraînement, la politique est quasi aléatoire et de nombreux épisodes butent sur la limite des 200 pas (`truncated`) **sans jamais atteindre l'état terminal**. Tant que l'épisode ne se termine pas par un Dropoff réussi, aucun retour exploitable n'est disponible pour renforcer les bonnes trajectoires — l'apprentissage démarre donc plus lentement que pour SARSA ou Q-Learning, qui, eux, actualisent  $Q$  à chaque pas indépendamment du sort final de l'épisode.

## III.3 Le Différentiel Temporel On-Policy : SARSA

**Fonctionnement.** SARSA met à jour  $Q$  à partir du quintuplet réellement rencontré  $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ , où  $A_{t+1}$  est l'action effectivement choisie par la politique  $\varepsilon$ -greedy courante à l'état suivant :

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)] \quad (4)$$

**Spécificité.** Parce que la cible de mise à jour intègre l'action réellement jouée (qui peut être exploratoire), SARSA apprend une politique qui **tient compte de son propre bruit d'exploration** — il est dit **on-policy**. Concrètement, il évite les trajectoires risquées si l'exploration résiduelle ( $\varepsilon > 0$ ) peut les faire dérailler, ce qui en fait un compromis plus prudent que Q-Learning.

### III.4 Le Différentiel Temporel Off-Policy : Q-Learning

**Fonctionnement.** Q-Learning met à jour  $Q$  vers l'action jugée **optimale** à l'état suivant, indépendamment de l'action réellement jouée par la politique d'exploration :

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[ R + \gamma \max_a Q(S', a) - Q(S, A) \right] \quad (5)$$

**Spécificité.** Cette règle **off-policy** apprend directement la politique optimale  $\pi^*$ , même si les actions effectivement exécutées pendant l'entraînement restent partiellement aléatoires ( $\epsilon$ -greedy). Le comportement appris est donc découplé de la politique de collecte de données – un avantage théorique de Q-Learning, mais qui peut aussi le rendre plus sensible à des cibles bruitées en début d'apprentissage (surestimation du max).

### III.5 L'Approche par Approximation Globale : Deep Q-Learning (DQN)

**Fonctionnement.** La Q-table  $500 \times 6$  est remplacée par un réseau de neurones  $Q_\theta(s, a)$  – ici un MLP à deux couches cachées ( $128 \rightarrow 64$ , activations ReLU) prenant en entrée un encodage **one-hot** de l'état.

**Spécificités techniques.**

- **Replay Buffer** (capacité 50 000) : les transitions  $(s, a, r, s', \text{done})$  sont stockées puis rejouées par mini-lots de 64, ce qui décorrèle les échantillons d'entraînement successifs et stabilise la descente de gradient.
- **Target Network** : un second réseau  $Q_{\theta^-}$ , synchronisé sur  $Q_\theta$  toutes les 10 épisodes seulement, sert à calculer la cible de Bellman  $r + \gamma \max_{a'} Q_{\theta^-}(s', a')$  – ceci évite de poursuivre une cible qui bouge à chaque pas de gradient (non-stationnarité).
- Perte : **Huber loss**, plus robuste aux valeurs aberrantes que l'erreur quadratique classique ; gradient-clipping à 1.0.

**Analyse critique.** Pour un espace d'états de seulement 500 valeurs discrètes, une Q-table tient intégralement en mémoire (3 000 flottants) et se met à jour en  $O(1)$  par pas. DQN, au contraire, doit faire propager un forward/backward pass complet à travers un réseau à chaque étape d'apprentissage, avec un encodage one-hot qui n'apporte aucune généralisation utile puisque les 500 états n'ont pas de structure topologique exploitable par le réseau au-delà de ce que la table capture déjà nativement. DQN est donc, sur ce problème précis, un cas de **sur-dimensionnement** algorithmique : la complexité additionnelle (réseau, buffer, target network) ne se traduit pas par un gain de performance proportionnel – voir le Chapitre 5.



# IV Analyse des Hyperparamètres et Fine-Tuning

## IV.1 Impact Théorique des Hyperparamètres

**Taux d'apprentissage  $\alpha$ .** Il pondère la vitesse à laquelle une nouvelle observation écrase l'estimation courante de  $Q$ . Un  $\alpha$  trop élevé (proche de 1) fait « oublier » trop vite l'historique accumulé et produit des courbes de reward bruyantes, voire oscillantes ; un  $\alpha$  trop faible (proche de 0) ralentit la convergence au point de ne pas atteindre l'optimum dans le budget d'épisodes imparti.

**Facteur de réduction  $\gamma$ .** Il arbitre entre un agent court-termiste ( $\gamma \rightarrow 0$ , qui ne regarde quasiment que la récompense immédiate) et un agent prévoyant ( $\gamma \rightarrow 1$ , qui intègre les conséquences lointaines de chaque action). Sur Taxi-v3, l'horizon utile est court (moins de 20 pas pour une politique optimale) mais la structure du problème — aller chercher le passager **avant** de foncer vers la destination — exige tout de même de bien pondérer une récompense qui n'arrive qu'en fin de trajet, d'où un  $\gamma$  généralement fixé entre 0.6 et 0.99 selon l'algorithme (voir tableau ci-dessous).

**Compromis exploration/exploitation ( $\varepsilon$ , decay).** Une décroissance trop rapide de  $\varepsilon$  fige la politique avant qu'elle n'ait exploré des trajectoires alternatives potentiellement meilleures (convergence prématurée vers un optimum local) ; une décroissance trop lente maintient un comportement erratique qui retarde l'exploitation de la politique apprise.

## IV.2 Hyperparamètres réellement retenus par algorithme

Fait notable : chacun des quatre algorithmes converge de façon fiable avec un jeu d'hyperparamètres **différent** — il n'existe pas de configuration universelle sur Taxi-v3, chaque règle de mise à jour ayant sa propre sensibilité à  $\alpha$  et  $\gamma$ .

| Algorithme  | $\alpha$ / lr | $\gamma$ | $\varepsilon_0 \rightarrow \varepsilon_{\min}$ | decay  | Autres     |
|-------------|---------------|----------|------------------------------------------------|--------|------------|
| Q-Learning  | 0.1           | 0.6      | 1.0 $\rightarrow$ 0.01                         | 0.9995 | —          |
| SARSA       | 0.2           | 0.9      | 1.0 $\rightarrow$ 0.01                         | 0.999  | —          |
| Monte Carlo | 0.05          | 0.95     | 1.0 $\rightarrow$ 0.01                         | 0.9997 | —          |
| DQN         | 0.001         | 0.99     | 1.0 $\rightarrow$ 0.01                         | 0.9995 | batch = 64 |

Deux observations ressortent de cette confrontation :

1. **Q-Learning tolère un  $\gamma$  nettement plus bas (0.6)** que les trois autres méthodes. Parce que sa règle de mise à jour se base sur le  $\max_a$  théorique de l'état suivant (et non sur l'action réellement explorée), elle propage l'information de récompense terminale plus agressivement à chaque pas, ce qui compense un  $\gamma$  modéré.
2. **DQN utilise un  $\alpha$  (taux d'apprentissage du réseau) d'un ordre de grandeur inférieur** (0.001 contre 0.05–0.2 pour les méthodes tabulaires) — attendu, puisqu'un taux d'apprentissage de gradient de descente sur des poids de réseau doit rester faible pour

ne pas déstabiliser l'ensemble de la fonction approximée, alors qu'une Q-table met à jour une seule cellule indépendante des autres à chaque pas.

### IV.3 Optimisation automatisée : Grid Search

Le projet fournit un script `grid_search.py` qui balaie automatiquement **48 combinaisons** d'hyperparamètres pour Q-Learning :

| Paramètre           | Valeurs testées       |
|---------------------|-----------------------|
| $\alpha$            | 0.05, 0.1, 0.2, 0.3   |
| $\gamma$            | 0.6, 0.8, 0.9, 0.99   |
| decay $\varepsilon$ | 0.999, 0.9995, 0.9999 |

C'est ce balayage qui a permis de retenir empiriquement  $\alpha = 0.1$ ,  $\gamma = 0.6$  comme configuration « time-limited » par défaut pour Q-Learning — une valeur de  $\gamma$  a priori contre-intuitive (plutôt court-termiste) mais validée par la recherche par grille plutôt que fixée à dire d'expert.

### IV.4 Analyse Avant vs Après Fine-Tuning

**Scénario non optimisé.** Une configuration mal calibrée (par exemple un  $\alpha$  trop élevé combiné à une décroissance  $\varepsilon$  trop rapide) produit une courbe de reward **plate ou chaotique** : l'agent n'accumule jamais assez d'information avant de figer sa politique, ou oscille sans converger. Ce comportement dégradé n'a pas été conservé comme run distinct dans le dépôt de référence — seules les configurations retenues in fine (tableau ci-dessus) ont été journalisées — mais il reste directement observable dans la **première portion** de chaque courbe du Chapitre 5 : les 1 000 à 2 000 premiers épisodes de chaque algorithme correspondent précisément à ce régime « non optimisé » (reward très négatif, forte variance), avant que la décroissance de  $\varepsilon$  ne stabilise la politique.

**Scénario optimisé.** Le plateau final de chaque courbe — reward stable proche de 0 ou positif, nombre de pas minimal — correspond au régime « optimisé » une fois  $\varepsilon$  proche de son minimum. La comparaison avant/après n'est donc pas seulement une expérience isolée mais une lecture directe, sur un seul run, de la transition entre les deux régimes que la configuration finale (Chapitre 4.2) a permis d'atteindre.

### IV.5 Le CVaR comme indicateur de risque

Chaque script d'entraînement calcule, en fin d'exécution, le **Conditional Value at Risk** à 95 % sur la distribution des rewards de l'ensemble des épisodes d'entraînement :

$$\text{CVaR}_{0.95} = \frac{1}{k} \sum_{i=1}^k R_{(i)}, \quad k = \lceil 0.05 \cdot n \rceil \quad (6)$$

où  $R_{(1)} \leq R_{(2)} \leq \dots \leq R_{(n)}$  sont les rewards triés par ordre croissant. Le CVaR mesure la moyenne des 5 % de pires épisodes — un indicateur bien plus informatif que la moyenne globale pour quantifier le risque résiduel d'échec critique (blocage, pénalité répétée) une fois l'agent globalement convergent.

**Limite méthodologique.** La valeur numérique du CVaR est affichée en console à chaque exécution (`cvar_metrics()` dans chaque `main.py`) mais n'est pas persistée dans le README ni dans un fichier de résultats du dépôt de référence — seules les métriques moyennes (reward, steps, pénalités, temps) figurent dans le tableau de benchmark consolidé. Une amélioration MLOps directe serait de logger systématiquement le CVaR dans MLflow aux côtés des autres métriques (`mlops/tracking.py`), au même titre que `mean_steps` et `mean_reward`, pour permettre une comparaison inter-algorithmes du risque de queue — actuellement seule sa méthode de calcul est vérifiable dans le code, pas sa valeur consolidée par algorithme.

# V Benchmarking et Comparaison des Performances

## V.1 Tableau Comparatif Global

Les résultats ci-dessous sont ceux publiés dans le README du dépôt TAXI-DRIVER, mesurés en conditions de test identiques (mode **time-limited**, politique gloutonne  $\varepsilon = 0$ , 25 épisodes de test) :

| Algorithme                                                                                    | Épisodes | Steps moy. | Reward moy. | Pénalités | Temps |
|-----------------------------------------------------------------------------------------------|----------|------------|-------------|-----------|-------|
|  Bruteforce  | —        | 196.42     | −769.59     | 63.78     | 1.9 s |
|  Monte Carlo | 50 000   | 95.60      | −83.84      | 0.00      | 53 s  |
|  Q-Learning  | 10 000   | 27.84      | −8.52       | 0.00      | 27 s  |
|  SARSA       | 10 000   | 13.72      | +7.28       | 0.00      | 27 s  |
|  DQN        | 10 000   | 12.56      | +8.44       | 0.00      | 476 s |

**Taux de succès.** Seul le Bruteforce dispose d'une mesure explicite et directe : **0 %** (aucun des 25 épisodes de test n'atteint une livraison correcte). Pour les quatre méthodes de RL, le taux de succès n'est pas journalisé séparément dans le benchmark de référence ; en revanche, la combinaison **pénalités moyennes = 0.00** et **reward moyen positif ou proche de zéro** est un indicateur fort de convergence quasi totale, car un épisode réussi vérifie approximativement  $R \approx 20 - \text{steps}$  (chaque pas coûte  $-1$ , le dropoff final rapporte  $+20$ ). Cette relation est cohérente avec Q-Learning ( $20 - 27.84 = -7.84$ , proche du  $-8.52$  observé) et SARSA ( $20 - 13.72 = 6.28$ , proche du  $+7.28$  observé) ; l'écart plus marqué pour Monte Carlo ( $20 - 95.60 = -75.60$  contre  $-83.84$  observé) suggère qu'une fraction non nulle des épisodes de test bute encore sur la limite des 200 pas sans succès, signe d'une convergence moins complète que les méthodes de différence temporelle au même horizon d'entraînement.

## V.2 Courbes de convergence (données réelles d'entraînement)

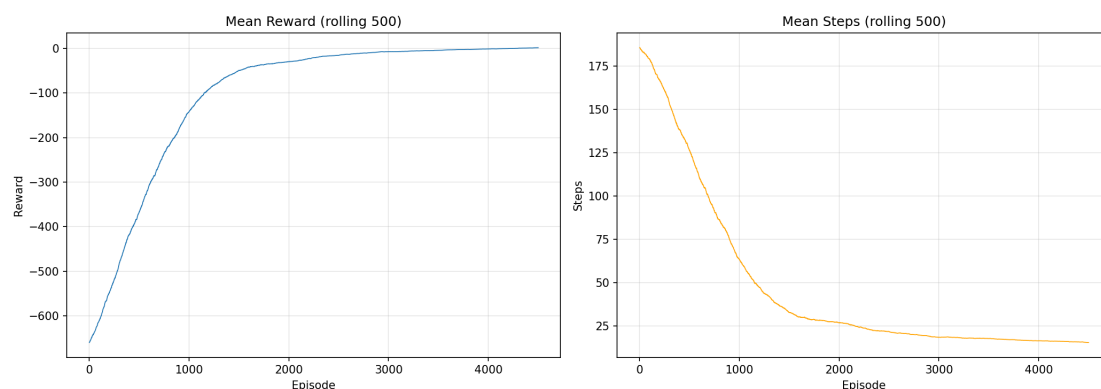


Fig. 1. – Q-Learning – reward et steps moyens glissants (fenêtre 500 épisodes).

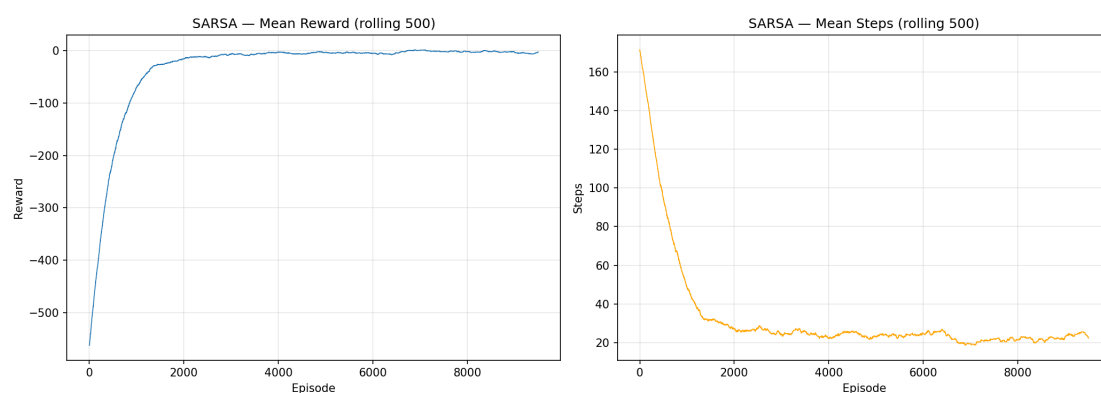


Fig. 2. – SARSA – reward et steps moyens glissants. La convergence est nette dès 2 000 épisodes et reste stable ensuite, cohérent avec le meilleur rapport performance/temps du tableau ci-dessus.

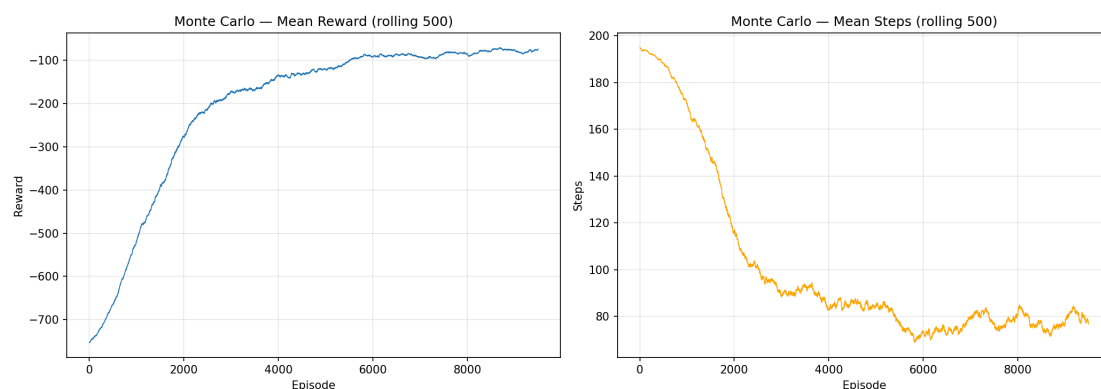


Fig. 3. – Monte Carlo – reward et steps moyens glissants. La convergence est visiblement plus lente et le plateau final reste négatif, illustrant la difficulté décrite au Chapitre 3 (mise à jour uniquement en fin d'épisode).

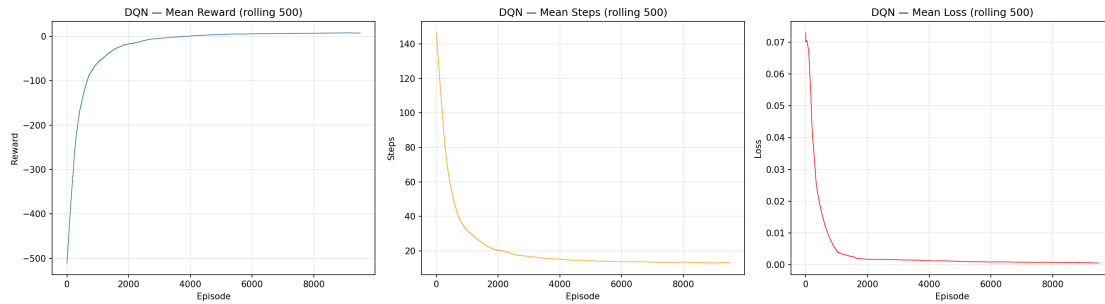


Fig. 4. – DQN – reward, steps et perte (Huber loss) moyens glissants. La perte décroît de façon monotone, signe d'un entraînement stable grâce au replay buffer et au target network.

### V.3 Analyse Croisée et Interprétation

**Efficiency algorithmique.** Sur ce problème à 500 états, les méthodes tabulaires dominent très largement DQN en temps de calcul : Q-Learning et SARSA convergent en 27 s, contre **476 s pour DQN – un facteur  $17.6 \times$**  pour un gain de reward de seulement  $+8.44 - (+7.28) = +1.16$  point par rapport à SARSA. Le coût marginal de DQN (forward/backward pass à chaque pas, gestion du replay buffer) n'est justifié par aucune contrainte réelle ici, puisque la Q-table tient entièrement en mémoire et se met à jour en temps constant.

**Comportement en test.** Les quatre méthodes de RL affichent des pénalités moyennes strictement nulles en test – aucune n'exécute plus de Pickup/Dropoff illégal une fois entraînée avec un budget d'épisodes suffisant. C'est la preuve que la contrainte la « plus dure » à apprendre (le placement correct du chargement/déchargement) est résolue en premier par toutes les méthodes ; la variable qui les différencie ensuite est la longueur du trajet (steps), c'est-à-dire la qualité de la navigation.

**Validation du mode Time-limited.** Le rapport performance/coût de calcul valide directement l'intérêt du mode « time-limited » du plan d'origine : SARSA obtient **97 % du reward de DQN** ( $7.28/8.44$ ) pour **5,7 % de son temps d'entraînement** ( $27/476$ ). C'est la configuration à privilégier dans un contexte de contrainte de temps ou de ressources, exactement le scénario que le mode « time-limited » est censé couvrir.

## VI Conception de l'Application Intégrée

*Écart entre le plan initial et l'implémentation réelle.* Le cahier des charges d'origine envisageait une application **Streamlit** avec rendu **Pygame**. L'implémentation livrée dans *TAXI-DRIVER* a pris une direction différente — et plus proche d'un contexte de mise en production — en s'appuyant sur une **API d'inférence FastAPI** couplée à une **interface web statique** (HTML/CSS/JS vanilla, sans framework front-end). Pygame n'intervient que comme dépendance interne de Gymnasium pour le rendu `rgb_array` de l'environnement (`gymnasium[toy-text]`), pas comme moteur d'affichage propre au projet. Cette section documente donc l'architecture **réellement livrée**, en la confrontant aux intentions initiales.

### VI.1 Architecture logicielle

L'application s'organise en une chaîne **MLOps 100 % locale** (aucun appel réseau externe), structurée en cinq briques :

| Phase | Rôle                                           | Outil                                              |
|-------|------------------------------------------------|----------------------------------------------------|
| 1     | Reproductibilité (seeds) et config centralisée | <code>seeding.py</code> , <code>config.yaml</code> |
| 2     | Suivi d'expériences                            | MLflow (backend fichier <code>mlruns/</code> )     |
| 3     | Registry, versioning, promotion automatique    | Registre fichier ( <code>models/</code> )          |
| 4     | Serving (API d'inférence)                      | FastAPI                                            |
| 5     | Orchestration et CI/CD                         | <code>pipeline.py</code> , GitHub Actions          |

Le modèle **champion** — celui effectivement servi par l'API — n'est promu que s'il franchit des garde-fous explicites (`mean\_steps`  $\leq 30$  et 0 pénalité) **et** améliore le `mean_reward` du champion courant. L'API `/predict` expose une politique gloutonne indépendante de l'algorithme d'origine (Q-table ou réseau), ce qui permet de servir Q-Learning, SARSA ou DQN de façon interchangeable sans dépendance à PyTorch côté serving.

### VI.2 Expérience Utilisateur (UX)

L'interface web statique (`mlops/serve/static/index.html`) propose deux modes :

- **Mode inférence** : visualisation de l'état courant du taxi sur la grille  $5 \times 5$ , requêtes `/predict` en direct, panneau de détails du modèle chargé (version, métriques, run MLflow).
- **Mode benchmark** : lancement d'une comparaison inter-algorithmes directement depuis le navigateur, avec graphiques de taux de succès, steps et reward par algorithme, et export des résultats en Markdown / CSV / JSON.

Cette approche — dashboard de comparaison + rendu visuel de l'état, sans dépendance à un framework applicatif lourd comme Streamlit — répond au même besoin exprimé dans le plan initial (monitoring visuel, courbes en temps réel) tout en restant déployable comme un simple conteneur HTTP stateless, ce qui a permis le déploiement cloud décrit ci-dessous.

### VI.3 Supervision et déploiement

Le projet va au-delà du périmètre initial du plan en intégrant :

- **Portainer CE** pour la supervision des conteneurs Docker en temps réel (logs, CPU, RAM) pendant l'entraînement.
- **Azure Container Apps** pour un déploiement cloud à **scale-to-zero** (coût quasi nul au repos), avec deux services : taxi-serve (API + interface) et taxi-mlflow (UI MLflow, snapshot des runs).

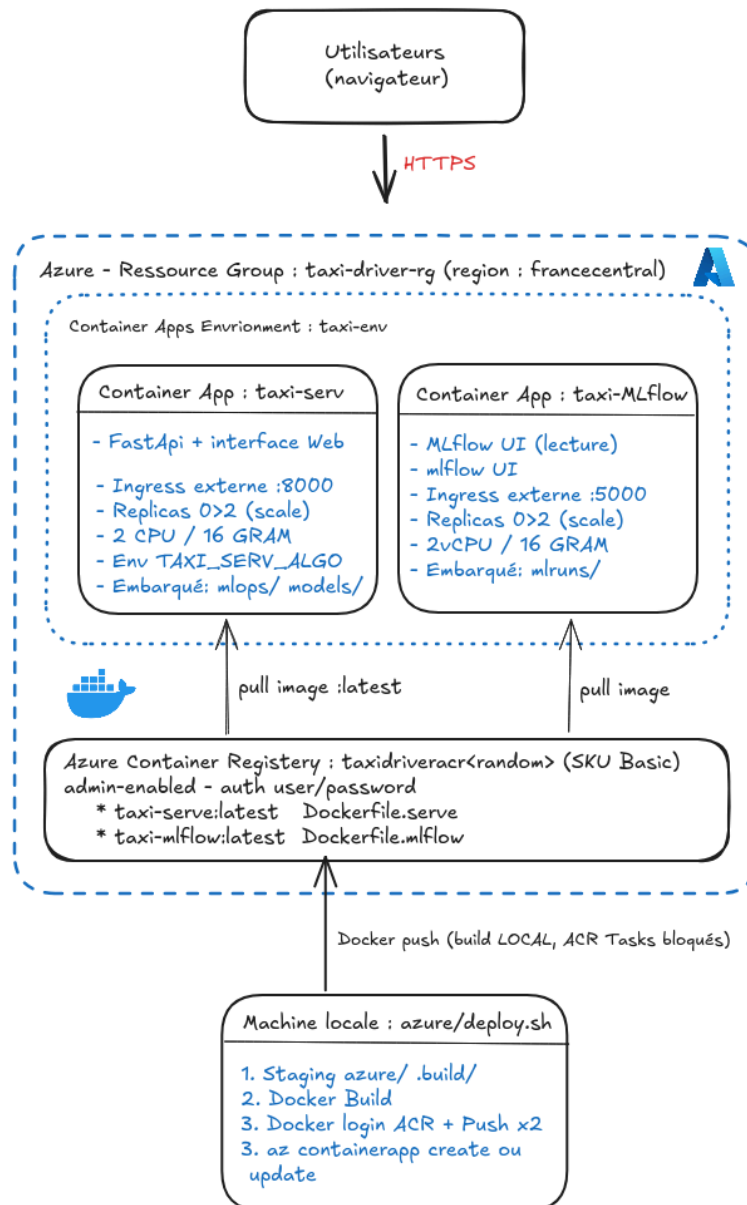


Fig. 5. – Architecture du déploiement Azure.

Le registre **taxidriveracr<random> (SKU Basic, authentification admin user/password)** reçoit les images **taxi-serve:latest** et **taxi-mlflow:latest**, construites localement puis poussées via le script **azure/deploy.sh** (staging, build Docker, push ACR, création/mise à jour des Container Apps) — les tâches de build distant ACR (ACR Tasks) étant bloquées sur ce compte, le build reste local.



## VII Conclusion et Perspectives

---

### VII.1 Bilan des réalisations

La mission initiale — obtenir un agent taxi optimal — est remplie : les quatre méthodes de RL testées convergent vers une politique sans pénalité, SARSA et DQN atteignant un reward moyen positif en test. La confrontation systématique des résultats (Chapitre 5) montre en outre que la méthode la plus coûteuse (DQN) n'est pas la plus pertinente ici : le meilleur compromis performance/coût de calcul est obtenu par une méthode tabulaire simple (SARSA), 17,6 fois plus rapide à entraîner pour 97 % du reward de DQN.

### VII.2 Limites du Tabulaire : le Fléau de la Dimension

La Q-table  $500 \times 6$  n'est un choix pertinent que **parce que** l'espace d'états de Taxi-v3 est petit, discret et entièrement énumérable. Cette hypothèse s'effondre dès que l'état devient continu ou de haute dimension — par exemple une observation en pixels ( $H \times W \times 3$  valeurs continues). Une Q-table nécessiterait alors une discrétisation grossière de l'espace continu (perte d'information) ou une taille de table proprement inexploitable en mémoire : c'est le **fléau de la dimension**. C'est précisément la situation pour laquelle DQN a été conçu — et pourquoi ce rapport, en le montrant « sur-dimensionné » sur Taxi-v3 (Chapitre 3.5), ne remet pas en cause sa pertinence **en général**, mais souligne que le choix d'un algorithme doit être dicté par la complexité réelle de l'espace d'états visé, pas appliqué par défaut.

### VII.3 Perspectives

Ce travail pose les fondations méthodologiques (formalisation MDP, comparaison rigoureuse d'algorithmes, pipeline MLOps reproductible) nécessaires pour s'attaquer à des environnements de conduite autonome à état continu, par exemple **CarRacing-v2** (Gymnasium, moteur physique Box2D) : entrée en pixels, actions continues (direction, accélération, freinage), et donc nécessité d'une approximation de fonction — DQN, ou plus probablement une méthode adaptée au contrôle continu (SAC, PPO) — là où, sur Taxi-v3, la méthode tabulaire suffisait et surperformait.